

003 -- Shock Event Analysis

Author: Wayne Kirk Schmidt
Email: wayne.kirk.schmidt@gmail.com

Purpose

`003_analysis.ipynb` is the third stage of the cryptocurrency statistical arbitrage research pipeline.

This notebook analyzes the statistical structure of return shocks across the crypto universe. It characterizes the distribution, magnitude, and co-occurrence of standardized return events -- providing the empirical foundation for the trading strategy defined in Stage 4.

This stage does not generate trading signals. All outputs are descriptive and support downstream hypothesis evaluation.

Inputs

From `output/002_enrich/`:

File	Description
<code>events_raw.pkl</code>	Unfiltered event panel
<code>events.pkl</code>	Synchronized event panel
<code>prices_raw.pkl</code>	Raw price series
<code>price_wide.pkl</code>	Wide-format close price matrix
<code>returns_full.pkl</code>	Synchronized daily return matrix
<code>rolling_sigma.pkl</code>	Rolling volatility estimates
<code>z_scores.pkl</code>	Standardized return matrix

Outputs

Written to `output/003_analysis/`:

File	Description
<code>sigma_event_matrix.pkl</code>	Event counts per asset at each sigma threshold
<code>extreme_z_scores.pkl</code>	Max/min z-scores and dates per asset
<code>shock_events.pkl</code>	Long-format enriched shock event dataset
<code>event_bitmap.pkl</code>	Dates and direction of simultaneous multi-asset shocks

Pipeline Position

```
001_download -> data acquisition
002_enrich   -> feature engineering
003_analysis <- you are here
-
004_strategy -> signal construction
-
005_backtest -> execution and performance evaluation
```

Notes

- No filtering or threshold selection is applied in this stage
- The shock event dataset preserves the full z-score distribution
- Sigma buckets 0 and 1 are present in the data but treated as noise in downstream stages

1. Imports and Environment Setup

Standard library imports for data manipulation, file I/O, and path handling.

```
In [1]: import pandas as pd
import numpy as np
import pickle
from datetime import datetime, timedelta, UTC
import math
from pathlib import Path
```

2. Environment Configuration

Defines the asset universe, pipeline parameters, and output directories for this stage.

```
In [2]: startdate = "2023-01-01"
trading_days = 252
frequency = "1d"

universe = [
    "BTCUSDT", # Bitcoin
    "ETHUSDT", # Ethereum
    "BNBUSDT", # Binance Coin
```

```
"SOLUSDT", # Solana
"XRPUSDT", # Ripple
"ADAUSDT", # Cardano
"DOGEUSDT", # Dogecoin
"AVAXUSDT", # Avalanche
"LTCUSDT"  # Litecoin
]

execution_delay = [0, 1, 2, 3]
execution_cost_bps = [20, 30, 40]

stage_label = "003_analysis"

OUTPUT_ROOT = Path("../output")
OUTPUT_ROOT.mkdir(parents=True, exist_ok=True)

MANIFEST_FILE = OUTPUT_ROOT / "manifest.pkl"

DOWNLOAD_DIR = OUTPUT_ROOT / "001_download"
DOWNLOAD_DIR.mkdir(parents=True, exist_ok=True)

ENRICH_DIR = OUTPUT_ROOT / "002_enrich"
ENRICH_DIR.mkdir(parents=True, exist_ok=True)

ANALYSIS_DIR = OUTPUT_ROOT / "003_analysis"
ANALYSIS_DIR.mkdir(parents=True, exist_ok=True)

inspection_window = 20

observation_window_length = 10
observation_window = range(1, observation_window_length + 1)

holding_period = 1
```

3. Load or Initialize the Manifest

Loads the shared pipeline manifest and initializes this stage's entry if not already present.

```
In [3]: MANIFEST_FILE = OUTPUT_ROOT / "manifest.pkl"

if MANIFEST_FILE.exists():
    manifest = pd.read_pickle(MANIFEST_FILE)
else:
    manifest = {}

manifest.setdefault(stage_label, {})
```

```
Out[3]: {}
```

4. Load Stage 2 Artifacts

Loads all enriched datasets produced by `002_enrich`. Shape diagnostics confirm each artifact loaded correctly before proceeding.

```
In [4]: EVENTS_RAW_FILE = ENRICH_DIR / "events_raw.pkl"
EVENTS_FILE = ENRICH_DIR / "events.pkl"
PRICES_RAW_FILE = ENRICH_DIR / "prices_raw.pkl"
PRICE_WIDE_FILE = ENRICH_DIR / "price_wide.pkl"
RETURNS_FULL_FILE = ENRICH_DIR / "returns_full.pkl"
ROLLING_SIGMA_FILE = ENRICH_DIR / "rolling_sigma.pkl"
Z_SCORES_FILE = ENRICH_DIR / "z_scores.pkl"

events_raw = pd.read_pickle(EVENTS_RAW_FILE)
events = pd.read_pickle(EVENTS_FILE)
prices_raw = pd.read_pickle(PRICES_RAW_FILE)
price_wide = pd.read_pickle(PRICE_WIDE_FILE)
returns_full = pd.read_pickle(RETURNS_FULL_FILE)
rolling_sigma = pd.read_pickle(ROLLING_SIGMA_FILE)
z_scores = pd.read_pickle(Z_SCORES_FILE)

print("events_raw:", events_raw.shape)
print("events:", events.shape)
print("price_wide:", price_wide.shape)
print("returns_full:", returns_full.shape)
print("rolling_sigma:", rolling_sigma.shape)
print("z_scores:", z_scores.shape)
```

```
events_raw: (74736, 5)
events: (74736, 5)
price_wide: (1242, 9)
returns_full: (1047, 9)
rolling_sigma: (1028, 9)
z_scores: (1028, 9)
```

5. Explore Sigma Thresholds Across Assets

To understand where statistically significant activity occurs, I count events at each sigma threshold (1σ through 5σ) for every asset in the universe.

This informs threshold selection in later stages. Observed event counts across the dataset (~971 trading days):

Sigma Threshold	Approx. Event Count	Approx. Frequency
$\geq 1\sigma$	~244	~25% of days
$\geq 2\sigma$	~62	~6% of days
$\geq 3\sigma$	~14	~1.4% of days
$\geq 4\sigma$	~3	~0.3% of days
$\geq 5\sigma$	0	none observed

Interpretation:

- **1 σ** events are too frequent and likely represent normal market noise
- **2 σ** events occur regularly and may contain both signal and noise
- **3 σ** events (~1 per 70 days) represent meaningful market shocks
- **4 σ** events (~1 per year) correspond to major market dislocations
- **5 σ** events (e.g. Lehman, COVID) were not observed in this sample

The most informative trigger range for cross-asset propagation analysis falls between **3 σ** and **4 σ** .

```
In [5]: sigma_levels = range(1, 6)

sigma_event_matrix = pd.DataFrame({
    coin: [(z_scores[coin].abs() >= s).sum() for s in sigma_levels]
    for coin in sorted(z_scores.columns)
}).T

sigma_event_matrix.columns = [f"{s}\u03c3" for s in sigma_levels]

# persist artifact
SIGMA_EVENT_MATRIX_FILE = ANALYSIS_DIR / "sigma_event_matrix.pkl"
sigma_event_matrix.to_pickle(SIGMA_EVENT_MATRIX_FILE)

# register artifact in manifest
manifest[stage_label][f"sigma_event_matrix"] = str(SIGMA_EVENT_MATRIX_FILE)

sigma_event_matrix
```

Out[5]:

	1 σ	2 σ	3 σ	4 σ	5 σ
ADAUSDT	290	62	9	1	0
AVAXUSDT	293	61	8	0	0
BNBUSDT	278	76	12	0	0
BTCUSDT	290	68	11	1	0
DOGEUSDT	270	63	11	0	0
ETHUSDT	285	64	13	2	0
LTCUSDT	269	59	15	0	0
SOLUSDT	314	62	8	0	0
XRPUSDT	268	68	17	2	0

6. Inspect Extreme Standardized Returns

Identifies the maximum and minimum z-score for each asset -- and the date on which it occurred. Sorted by largest absolute shock magnitude, this gives a quick view of which assets experienced the most extreme movements in the sample.

```
In [6]: extreme_table = pd.DataFrame({
    "max_z": z_scores.max(),
    "max_date": z_scores.idxmax(),
    "min_z": z_scores.min(),
    "min_date": z_scores.idxmin()
})

# largest absolute shock magnitude
extreme_table["largest_abs"] = extreme_table[["max_z", "min_z"]].abs().max(axis=1)

# deterministic ordering
extreme_table = extreme_table.sort_values("largest_abs", ascending=False)

# persist artifact
EXTREME_Z_FILE = ANALYSIS_DIR / "extreme_z_scores.pkl"
extreme_table.to_pickle(EXTREME_Z_FILE)

# register artifact in manifest
manifest[stage_label][f"extreme_z_scores"] = str(EXTREME_Z_FILE)

extreme_table
```

Out[6]:

	max_z	max_date	min_z	min_date	largest_abs
coin					
ADAUSDT	4.281478	2025-03-02 00:00:00+00:00	-3.731054	2025-10-10 00:00:00+00:00	4.281478
ETHUSDT	4.040426	2023-11-09 00:00:00+00:00	-4.155917	2023-08-17 00:00:00+00:00	4.155917
BTCUSDT	3.741367	2023-10-23 00:00:00+00:00	-4.079907	2023-08-17 00:00:00+00:00	4.079907
XRPUSDT	3.867720	2025-03-02 00:00:00+00:00	-4.044816	2026-02-05 00:00:00+00:00	4.044816
BNBUSDT	3.841387	2024-12-03 00:00:00+00:00	-3.440522	2023-08-17 00:00:00+00:00	3.841387
AVAXUSDT	3.318753	2024-03-11 00:00:00+00:00	-3.811970	2025-10-10 00:00:00+00:00	3.811970
DOGEUSDT	3.805705	2024-02-28 00:00:00+00:00	-3.393044	2025-10-10 00:00:00+00:00	3.805705
LTCUSDT	3.453240	2025-07-19 00:00:00+00:00	-3.745821	2025-10-10 00:00:00+00:00	3.745821
SOLUSDT	3.642370	2023-11-10 00:00:00+00:00	-3.586782	2025-02-24 00:00:00+00:00	3.642370

7. Build the Enriched Shock Event Dataset

Converts the wide z-score matrix into a long-format dataset where each row represents one (date, coin) observation. Adds:

- `abs_z` -- absolute magnitude of the standardized return
- `sigma_bucket` -- integer floor of `abs_z`, capped at 5
- `shock_sign` -- direction of the move (+1 up, -1 down)

The full distribution is preserved without filtering; threshold selection is deferred to downstream stages.

```
In [7]: # Convert wide z_scores (date x coin) -> Long format
z_long = z_scores.stack().reset_index()
z_long.columns = ["event_date", "coin", "z_score"]

# Absolute sigma
z_long["abs_z"] = z_long["z_score"].abs()

# Keep FULL distribution (no filtering)
shock_events = z_long.copy()

# Sigma bucket (integer levels: 0,1,2,3...)
shock_events["sigma_bucket"] = shock_events["abs_z"].astype(int)

# Optional: cap extreme buckets (keeps things sane later)
shock_events["sigma_bucket"] = shock_events["sigma_bucket"].clip(upper=5)

# Shock direction
shock_events["shock_sign"] = shock_events["z_score"].apply(
    lambda x: 1 if x > 0 else -1
)

# --- Sort for consistency ---
shock_events = shock_events.sort_values(
    ["event_date", "coin"]
).reset_index(drop=True)

# --- Diagnostics ---
print("Shock events shape:", shock_events.shape)
print("Unique dates:", shock_events["event_date"].nunique())
print("Events per coin:")
print(shock_events["coin"].value_counts())

print("\nSigma bucket distribution:")
print(shock_events["sigma_bucket"].value_counts().sort_index())

# Preview
shock_events.head()

SHOCK_EVENTS_FILE = ANALYSIS_DIR / "shock_events.pkl"
shock_events.to_pickle(SHOCK_EVENTS_FILE)
manifest[stage_label]["shock_events"] = str(SHOCK_EVENTS_FILE)
print("saved:", SHOCK_EVENTS_FILE)
print("shape:", shock_events.shape)
```

Shock events shape: (9252, 6)

Unique dates: 1028

Events per coin:

```
coin
ADAUSD  1028
AVAXUSD  1028
BNBUSDT  1028
BTCUSD  1028
DOGEUSD  1028
ETHUSD  1028
LTCUSD  1028
SOLUSD  1028
XRPUSD  1028
```

Name: count, dtype: int64

Sigma bucket distribution:

```
sigma_bucket
0      6695
1      1974
2        479
3         98
4          6
```

Name: count, dtype: int64

saved: ..\output\003_analysis\shock_events.pkl

shape: (9252, 6)

8. Build the Multi-Asset Shock Event Bitmap

Identifies dates where at least two assets simultaneously experienced a shock at or above the 3 σ threshold. For each such date, records the direction (+ or -) of each asset's shock and the total count of shocked assets.

This bitmap is used in Stage 4 to identify leader-follower relationships; when multiple assets spike together, the first mover is the candidate leader.

```
In [8]: threshold = 3
rows = []

for date, row in z_scores.iterrows():

    shock_mask = row.abs() >= threshold

    if shock_mask.sum() >= 2:

        record = {"date": date, "count": int(shock_mask.sum())}

        for coin in z_scores.columns:
            if shock_mask[coin]:
                record[coin] = "+" if row[coin] > 0 else "-"
            else:
                record[coin] = "."
```

```
rows.append(record)

event_bitmap = pd.DataFrame(rows)

# deterministic column order
event_bitmap = event_bitmap[
    ["date", "count"] + sorted(z_scores.columns)
]

# sort by largest cross-asset shock
event_bitmap = event_bitmap.sort_values("count", ascending=False)

# persist artifact
EVENT_BITMAP_FILE = ANALYSIS_DIR / "event_bitmap.pkl"
event_bitmap.to_pickle(EVENT_BITMAP_FILE)

# register artifact
manifest[stage_label]["event_bitmap"] = str(EVENT_BITMAP_FILE)

event_bitmap.head(20)
```

Out[8]:

	date	count	ADAUSD	AVAXUSD	BNBUSDT	BTCUSD	DOGEUSD	ETHUSD	LTCUSD	SOLUSD	XRPUSD
18	2026-02-05 00:00:00+00:00	9	-	-	-	-	-	-	-	-	-
0	2023-08-17 00:00:00+00:00	7	-	-	-	-	.	-	-	.	-
14	2025-05-08 00:00:00+00:00	6	+	+	.	.	+	+	.	+	+
15	2025-10-10 00:00:00+00:00	5	-	-	.	.	-	.	-	.	-
12	2025-03-02 00:00:00+00:00	4	+	.	.	+	.	.	.	+	+
11	2025-02-24 00:00:00+00:00	4	.	.	.	-	-	-	.	-	.
7	2024-07-04 00:00:00+00:00	4	-	.	-	.	.	-	.	.	-
13	2025-04-06 00:00:00+00:00	3	-	.	.	.	.	-	-	.	.
4	2024-03-11 00:00:00+00:00	3	.	+	.	.	.	.	+	.	+
2	2023-11-10 00:00:00+00:00	2	.	+	.	.	.	.	.	+	.
1	2023-10-23 00:00:00+00:00	2	.	.	.	+	+	.	.	.	.
10	2025-01-15 00:00:00+00:00	2	.	.	.	.	.	.	+	.	+
9	2024-11-06 00:00:00+00:00	2	.	.	.	+	.	+	.	.	.
8	2024-08-05 00:00:00+00:00	2	.	.	.	.	.	-	-	.	.
6	2024-05-20 00:00:00+00:00	2	.	.	.	.	.	+	+	.	.
3	2024-02-28 00:00:00+00:00	2	.	.	.	+	+	.	.	.	.
5	2024-04-12 00:00:00+00:00	2	-	.	.	.	.	.	.	.	-
16	2025-11-03 00:00:00+00:00	2	.	.	.	.	.	.	-	-	.
17	2026-01-13 00:00:00+00:00	2	.	.	.	+	.	+	.	.	.

999. Persist the Manifest

Records all artifact paths and a completion timestamp for this stage into the shared manifest.

In [9]:

```
manifest[stage_label]["timestamp"] = datetime.now(UTC).isoformat()
pd.to_pickle(manifest, MANIFEST_FILE)
print("manifest saved:", MANIFEST_FILE)
print("pipeline stages:", sorted(manifest.keys()))
print(f"artifacts in {stage_label}:", sorted(manifest[stage_label].keys()))

manifest saved: ..\output\manifest.pkl
pipeline stages: ['001_download', '002_enrich', '003_analysis']
artifacts in 003_analysis: ['event_bitmap', 'extreme_z_scores', 'shock_events', 'sigma_event_matrix', 'timestamp']
```

In []:

In []: